Inner Sydney Transit Projection Model — Project Report and Handover

As of: Thursday 5 November 2026, 11 pm (Sydney), the night before the showcase
Showcase: Friday 6 November 2026
Feature freeze: 30 October 2026. Every change after it was at the owner's request; §0 lists the ones made on 5 November
Build: branch `week13`, pushed to GitHub (`git log -1` for the exact commit), 173 automated tests passing
Author: Yoseph Kim, with Claude Code
Name: formerly *Pulse CBD*, renamed on 5 November 2026. The folder, code, service and log names still use `pulse-cbd` / `pulsecbd`

How to read this report

This report is written for someone who has never seen the project: a teammate picking it up, an assessor, or a new Claude Code session with no memory of earlier work. It assumes no knowledge of the code, of Transport for NSW's data, or of the decisions made along the way. Every technical term is explained the first time it matters, and §3 is a glossary to come back to.

It is organised in five parts:

- **§0 — Handoff**: the state as of 5 November, 11 pm, what changed that day, and what's left. Start here.
- **Part I — Understanding the project** (§1–3): what the project is, what someone standing at the model sees, the course and physical setup it was built for, and the ideas needed to follow the rest.
- **Part II — How it works** (§4–8): the parts of the system and how data moves through them, every data source, every feature, exactly how every number is calculated, and what every colour means.
- **Part III — Why it is built this way** (§9–11): the main design decisions and the alternatives rejected, the analysis comparing our own congestion measure with Transport for NSW's, and how the system was tested.
- **Part IV — Running it** (§12–14): how to start, stop, record, calibrate and troubleshoot it, the code repository, and the exact state of the laptop today.
- **Part V — Assessment** (§15–20): limitations to state honestly in the final report, remaining work, suggestions, open decisions, the timeline, and a map of the key files.

The appendices list every display setting, every server address and every configuration value. The full reasoning behind each decision, in the order decisions were made, is in `docs/DECISIONS.md`; this report summarises it.

For a new Claude Code session, the essentials:

Thing	Where
Project folder	`~/pulse-cbd` (Python 3.13 virtual environment in `.venv`; activate with `source .venv/bin/activate`)
Branch the laptop runs	`week13`, pushed to GitHub (latest commit: `git log -1`). See §13 for how it relates to `w12merge`
GitHub remote	`group-repo` → `https://github.com/0rangeGoose/Civil3704-group-6` (private)

Thing	Where
Decision log	`docs/DECISIONS.md` (newest first)
How to run everything	`README.md`
Original project brief	`~/Downloads/pulse-cbd-claude-code-prompt.md`

The owner's working rules (from `~/.claude/CLAUDE.md` and instructions given in sessions):

- Ask before any git commit unless told to commit. New work goes to `week13` (since 5 Nov evening; before that `w12merge`). **Never push to** `integration` without an explicit instruction.
- Check the changes for the TfNSW API key (kept in `.env`) before every push.
- Use the `gh` command-line tool for GitHub.
- When instructions are unclear or conflicting, stop and ask; never override or skip an instruction without asking.
- Explain in plain language; the owner asks "what do you mean by…" when jargon slips in.
- After changing a page, check it in Chrome; after changing the server, restart it (`launchctl kickstart -k gui/$(id -u)/com.pulsecbd.server`). Open pages need a reload to pick up new code.

```
cd ~/pulse-cbd && git status -sb && git log --oneline -3
lsof -nP -iTCP:8765 -sTCP:LISTEN           # is the server running?
curl -s http://127.0.0.1:8765/health        # vehicles, feed age, API quota use, playback mode
tail -5 data/recordings/recorder.log        # scheduled peak recordings
source .venv/bin/activate && pytest -q      # 173 tests should pass
```

---

0. Handoff: where things stand (5 November, 11 pm)

Read this first if you are a new Claude Code session picking the project up.

State right now

- **Code:** everything below is committed and pushed to the new branch `week13` on GitHub, by the owner's choice, *without* merging anything from other branches. In particular, the teammate's "gradient" commit `3b32562` (pushed to `w12merge` at 9:22 pm on 5 Nov) is **not** in `week13`. It blends vehicle lateness colours from green to amber to red, and stop-history delay colours too, but it was written for the old 2× very-late rule and conflicts with the new punctuality code. Combining the two would mean adapting the gradient to the sourced limits. Ask the owner first. `git status` should be clean apart from the untracked teammate file `pulse-cbd-context.md`, if it reappears.
- **The server** (launchd job `com.pulsecbd.server`) is running that code. It was last restarted at 10:45 pm on 5 Nov, live, with about 150–490 vehicles depending on the hour.
- **The pages** load fresh code from disk. Any projector or tablet page opened before 10:45 pm needs a **reload**.
- **Recordings:** the Thursday 5 Nov AM and PM peaks are both recorded at 100%, giving 28 replayable sessions.
- **The data release** on GitHub (`data-2026-11-05`, 32 files, 27.7 GB) holds everything recorded up to about 6:20 pm on 5 Nov. Today's PM peak isn't in it.
- **Tests:** `pytest -q` → 173 passed.

What changed on 5 November (each with a dated entry in `docs/DECISIONS.md`)

1. **Replays:**

    - Replay mode button, then a day dropdown and AM/PM peak: every recorded day is selectable.
    - Only one Time button is lit at a time.
2. **Legend:** the tablet's "What the colours mean" defines every colour with its cut-off, read from the server (§8).
3. **Merged the teammate's corner fit** (Team → Alignment, **F**) and README rewrite.
4. **Removed:**
    - the interchange "breathing" rings;
    - the team-tab PIN;
    - Opal 2016;
    - the Pressure (combined) layer;
    - **the street colours, along with their engine code, BOAM and the step 6 analysis code**.

    Each is summarised in its old section, which the report keeps so section numbers stay valid.
5. **Renamed** Pulse CBD to **Inner Sydney Transit Projection Model** wherever people see the name. The folder, code and service names still say `pulse-cbd`.
6. **The tablet fits phone screens** (320–1280 px).
7. **Punctuality rebuilt from published standards** (§7.2):
    - Sydney Trains ≤ 5 min, intercity ≤ 6 min (TfNSW);
    - buses from 59 s early to 5 min 59 s late (bus contract), with a new **Early** state;
    - Metro, light rail and ferries ≤ 5 min (borrowed from the rail standard, and labelled as such);
    - "very late" becomes **missed its slot** (later than the scheduled gap to the next service);
    - the Stats tab counts **each trip once at TfNSW's measuring points**.
8. **Early buses switch** (Display → Colour the vehicles by, or **E**): shows or hides the blue Early colour. On by default.
9. **CSV export of any recorded peak** (Display → Time → Download a recorded peak, or `scripts/export_peak.py`): vehicles, stop visits, trips on time (§12.9, Appendix D).
10. **Hover highlight:** pointing at a vehicle or stop on the projector page highlights its route(s) (§6.1).
11. **Data:** everything in `data/` was uploaded to a private GitHub release (§13).

Still to decide or do before the showcase (details in §16)

- **Mouse pointer:** the projector page is meant to hide it, but the teammate disabled that on 29 Oct for testing. Hover needs a pointer, so the owner must choose.
- **The physical model:** every layer still needs checking on it (alignment was done by the teammate). Test the networking (hotspot versus Wi-Fi). Agree with the teammate that `week13` is the showcase build, and what to do about their gradient commit on `w12merge`.
- **Showcase replay:** pick the default (e.g. Wed 4 Nov AM or Thu 5 Nov AM, both 100%). Remember to press **Live now** after each demonstration.

How the owner likes to work: see the working rules below. In particular, ask before committing unless told to commit, verify with the owner before removing features, and check pages in Chrome after changes.

Part I — Understanding the project

1. What the Inner Sydney Transit Projection Model is

1.1 In one paragraph

The Inner Sydney Transit Projection Model turns a physical scale model of inner Sydney into a live map of its public transport. A projector mounted above the model shines an image straight down onto it. That image shows every train, metro, light rail vehicle, ferry and bus currently inside the model's area — typically 270 to 490 of them at once (487 at 4:27 pm on 5 November) — as a small moving arrow on the street or track it is really on, updated every 15 seconds from Transport for NSW's live data. The vehicles' colours show how late each one is, how congested its traffic is or how full it is; stops can be coloured by their typical delay; and area layers show where lots of jobs are served by little public transport. One peak hour can be compared with another. It can show what is happening now, or replay any recorded morning or evening peak as a sped-up time-lapse. Visitors control it from a tablet beside the model; the projected image itself contains only map content.

1.2 What a visitor experiences

Someone walking up to the model during the showcase would see, in a typical demonstration:

1. **The network moving.** Hundreds of arrows travel along the model's streets and rail lines. Bigger arrows are trains, metro and light rail; smaller ones are buses and ferries. Each leaves a short fading tail so its movement is visible even between updates. By default each arrow is coloured by punctuality, using Transport for NSW's own on-time standards: blue early (buses only), green on time, amber late, red when it has missed its slot (so late that the next service would have come sooner) or is cancelled, and grey when there is no schedule information to compare with.
2. **A peak hour in five minutes.** The team taps *Replay mode* on the tablet, picks a day and *AM peak*. The model jumps to 7:00 that morning and plays it back 25 times faster than real time, so two hours of the morning rush pass in under five minutes. The presenter can pause, jump forwards or back, and point out where buses bunch up.
3. **Coloured areas.** Turning on an area layer shades the whole model in patches — either Transport for NSW's "travel zones" or a regular 150 m grid — by how many jobs are there, or by how well-served those jobs are by public transport (the *service gap*). Each area layer stands on its own.
4. **Stops with a history.** *Stop markers* show every bus, train, light rail and ferry stop on the model. Turning on *Stop history* colours each stop by how late vehicles typically are when they reach it (median or mean), or, with the crowding colours on, by how often they arrive with few seats or fuller. It draws on every stop visit recorded live since 30 October — 865,907 by the afternoon of 5 November — and a stop needs at least 20 visits before it is coloured. The teammate's notes (as of 4 November) report that at 610 of 662 stops (92%) crowded vehicles arrived later than seated ones: an association, not proof that crowding causes delay. A stop's colour shows lateness *inherited* from earlier in the trip, not only delay caused at that stop (§7.13).
5. **One route at a time.** Typing route names into the *route filter* — for example 333, T1 or L2, up to eight at once — makes those routes' vehicles, line and stops stand out, each in its own fixed colour, while everything else fades back. It is the easiest way to follow one bus route through the peak.
6. **A map underneath, on a screen.** The model already has streets, so the background is plain by default. For a laptop or flat-screen demonstration, *Background map* adds faint roads, rail, water and parks (*Detailed*), or those plus suburb, station, landmark and street names (*Full*).
7. **A legend that explains everything.** The tablet's *What the colours mean* section explains every colour currently shown, with the exact numbers behind it — for example, "On time: up to 5 min behind the timetable for trains, buses and ferries; 2 min for metro and light rail".

1.3 What it is for

The course asks for a transport planning tool, and the model is the canvas. Its purpose is to make the network's performance *visible in place*: punctuality, congestion, crowding and access shown on the actual streets and districts of a model people can walk around, rather than in charts. It answers questions such as:

- Where are buses delayed during the morning peak, and does it match where Transport for NSW says traffic is congested?
- Which stops do vehicles typically reach late or crowded?
- Which parts of the inner city have many jobs but few nearby public transport departures?
- How does today's peak compare with last Tuesday's?

1.4 Status at a glance

The project was planned on 25 September as seven steps. Six are finished; the seventh needs the physical model.

Step (agreed 25 Sep)	What it means	Status
1. Footprint, projector view, tablet controller, scheduled recording	The basic system: the right map area, the projected image, a tablet to control it, automatic recording of peaks	Done
2. Traffic and crowding on streets	Colour streets by Transport for NSW's live traffic and crowding fields	Built and improved (pooling, directions, 4 Nov); **removed 5 Nov** at the owner's request (§7.5)
3. Replay and time-lapse	Play back recorded peaks, sped up	Done; pause and scrub added 30 Oct, any-day picker 5 Nov
4. Jobs, service gap, Opal 2016	Area layers from employment, timetable and ticketing data	Done; the Opal 2016 layer was removed on 5 Nov (§7.7)
5. Combined "pressure" score	One area score combining several inputs, with adjustable weights	Built, then removed on 5 Nov at the owner's request (§7.8)
6. Our own congestion measure, compared with Transport for NSW's	An independent measure we can explain, and a statistical comparison	Done; agreement turned out weak. Its code was removed with the street colours on 5 Nov; the results stay in §10
7. Testing on the model, report figures, rehearsal	Calibrate onto the real model and rehearse the showcase	**Partly.** The teammate's 5 Nov README says the system is running on the physical model, aligned with their new corner fit (§12.4). Report figures and a rehearsal aren't recorded (§16)

Beyond the plan, the project also has: polling that automatically stays within the daily data quota; a refreshed timetable; a teammate's stop history, route filter, direction arrows and corner fit; an optional background street map; full colour definitions on the tablet; punctuality from published TfNSW standards with per-trip figures; CSV export of any recorded peak; and route highlighting on hover. Several features were built and later removed at the owner's request (street colours, BOAM, pressure, Opal 2016, interchange rings), see §0.

- **Tests:** 173 automated tests run in about 2 seconds without internet access.
- **Data recorded:** 17 weekday peaks (16 of them at least 98% complete), plus weekends, nights and middays: 28 replayable sessions in total.
- **Biggest risks:** not every layer has been checked on the model (alignment has, per the teammate); the showcase network setup is untested (§16); a Wi-Fi drop during a live demo would freeze the vehicles; the mouse pointer currently shows on the projection.

---

2. Background

2.1 The course and the team

This is the CIVIL3704 group project for Group 6, with a final showcase on **Friday 6 November 2026**, where the team presents to assessors and visitors then try the installation themselves. Team roles were never formally divided.

Two pieces of software came out of the team:

- **This project** (then called Pulse CBD), built by the owner, Yoseph Kim, starting as a prototype in August.
- **A separate web dashboard** built by teammates (branches `mid-project-demo`, `test-pipline`), which computed its own congestion and demand scores on a map.

On 25 September the team agreed to make this project the single codebase, keep the dashboard's *direction* (a planning tool about congestion, demand and peak comparison), and rebuild its calculations, which had flaws (§9.1). Teammate **0rangeGoose** later added stop history, fixes to how delays are read, a route filter and direction arrows to this codebase itself (29–30 Oct); that work is merged in (§13).

2.2 The physical setup

- **The model.** A tabletop scale model of Sydney's inner city. The team surveyed its corners: north-east –33.835123, 151.275993 and south-west –33.894723, 151.132966. That is about **13.2 km east–west by 6.6 km north–south**, roughly two to one. This rectangle is called the **footprint**, and it is set in `config/config.yaml` as `bounding_box`. Everything outside it is ignored, because anything drawn there would land on the table rather than the model.
- **The projector.** Mounted above the model and **mirroring the laptop's screen**. This one fact shaped the whole design: anything on the laptop screen — buttons, the mouse pointer, notification pop-ups — would be projected onto the model. So the projected page shows map content only, and all controls live on a separate tablet (§9.2).
- **Access.** The rig stays set up between weekly sessions, so calibration (lining the image up with the model) mostly needs small corrections rather than starting over.
- **Users.** The team presents; visitors then use the tablet themselves.

2.3 How the plan was set

On 25 September the owner and Claude Code worked through the design one question at a time (a "grilling" session; recorded in `docs/DECISIONS.md`, 25 Sep). The outcomes that shaped everything since:

- One codebase (this one, then Pulse CBD), with the teammates' planning direction but rebuilt calculations.
- The footprint from the team's surveyed corners, replacing an earlier 5 × 4 km placeholder box.
- A projection with map content only; controls on a tablet, with a visitor view, a team panel (PIN-locked until 5 November) and a statistics page; the tablet connects over the laptop's own Wi-Fi hotspot; the laptop keyboard as a backup.
- Congestion first from Transport for NSW's own label, then from a measure of our own, checked against it.
- Demand layers — live crowding, jobs, service gap, Opal 2016 (removed 5 Nov) — each switchable, on both travel zones and a grid, plus a combined pressure layer (removed 5 Nov).

- Replays of recorded AM and PM peaks as the showcase default, with weekday peaks recorded automatically.
- No background map, because the model already has streets. Partly reversed on 4 November with an optional one that is off by default (§6.1).

---

3. Key ideas and glossary

The rest of the report uses these terms. Each is explained here once.

3.1 Transport data

- **Transport for NSW (TfNSW).** The NSW government agency that runs Sydney's public transport. It publishes data through its **Open Data Hub**, which needs a free API key (a password-like code identifying who is asking for data). The project's key is in the file `.env` and is never committed to the code repository.
- **GTFS (General Transit Feed Specification).** The worldwide standard format for timetables. A GTFS "bundle" is a set of tables: **routes** (e.g. bus 333), **trips** (one run of a route, e.g. the 7:42 am 333 to Bondi), **stops**, **stop times** (when each trip is scheduled at each stop), and **shapes** (the line on the map a trip follows). The project downloads TfNSW's complete bundle, keeps only what touches the footprint, and stores it in a SQLite database (a single-file database). This is called the **static** timetable because it changes only when the timetable does.

- **GTFS-Realtime.** The companion live format. TfNSW publishes three kinds of live feed for each of seven modes (Sydney Trains, NSW TrainLink intercity, Metro, two light rail lines, buses, ferries):

    - **Vehicle positions:** where each vehicle is, which trip it is running, and optional extra fields including `congestion_level` (how congested the traffic around it is) and `occupancy_status` (how full it is).
    - **Trip updates:** predicted delays at upcoming stops — for example "this trip will reach its next stop 4 minutes late".
    - **Service alerts:** disruptions and notices.

    TfNSW adds its own extension (field number 1007) to report crowding per train carriage.

- **Delay.** In this project, a vehicle's delay is TfNSW's predicted delay at the vehicle's **next stop**, in seconds. Positive means late. It comes from trip updates, not from comparing the vehicle's position with the timetable ourselves.

- **Peak.** The busy commuting periods. Throughout the project, the **AM peak** is 7:00–9:00 and the **PM peak** 16:30–18:30, Sydney time, on weekdays.

3.2 Places and demand data

- **Travel zones (TZ21).** TfNSW divides NSW into small areas called travel zones (the 2021 version is TZ21) for its transport models. 405 of them touch the footprint. They vary greatly in size: tiny in the CBD, much larger in the suburbs.
- **TZP24 employment.** TfNSW's travel zone projections (2024 release) estimate how many jobs are located in each zone, by year. The project uses the 2026 estimate. It counts people *working* in a zone, not living there, and it is a projection, not a census count.

- **Opal.** Sydney's ticketing smartcard. Passengers "tap on" when boarding and "tap off" when leaving. Until 5 November the project used Opal-based data in two ways, a layer of anonymised 2016 tap counts and TfNSW's BOAM bus loads; both were removed (§7.7, §5.6).

3.3 Methods and statistics

- **Map matching.** GPS positions wander off the road. Map matching moves ("snaps") each vehicle's reported position onto the nearest point of the line its trip follows, so it is drawn on the street, not beside it.
- **Level.** TfNSW's traffic and crowding fields are categories, not measurements. The project turns each into a number from 0 to 3 (0 = smooth or plenty of seats, 3 = severe or full), which sets a vehicle's colour.
- **Free-flow speed.** How fast buses move along a street when it is empty. The project estimates it from night-time recordings (§7.10).
- **Travel-time index (TTI).** Free-flow speed divided by current speed: how many times longer a trip takes now than when the street is empty. 1.0 means as fast as at night; 2.0 means twice as long.
- **85th percentile.** The value that 85% of readings are at or below. Used for free-flow speed because the very fastest readings may be errors, while the median would include slow readings caused by stops and lights.
- **Percentile rank.** Where a value sits among all values, from 0 (lowest) to 1 (highest). An area with more jobs than 90% of areas has a percentile rank of 0.9. Ranks have no units, so very different quantities (jobs, departures, taps) can be compared or combined.
- **Fifths (quintiles).** Sorting areas by a value and splitting them into five equal-sized groups. The colour then says "this area is in the top fifth", not "this area has more than X".
- **Cohen's weighted kappa (κ).** A score for how well two sets of categories agree, after removing the agreement expected by chance. 0 means no better than chance, 1 means perfect agreement; "weighted" means near-misses (one level apart) count partly. It matters because two measures that both say "smooth" most of the time will often match by coincidence.
- **Spearman's rank correlation (ρ).** Whether two measures rise and fall together, using their ranks; from −1 (opposite) through 0 (unrelated) to 1 (same order).
- **Homography.** The mathematical transform that maps a flat map onto a flat surface seen at an angle. Calibration computes one from at least four known points, so map coordinates land on the right spot of the model even if the projector isn't perfectly square to it.

3.4 Software terms

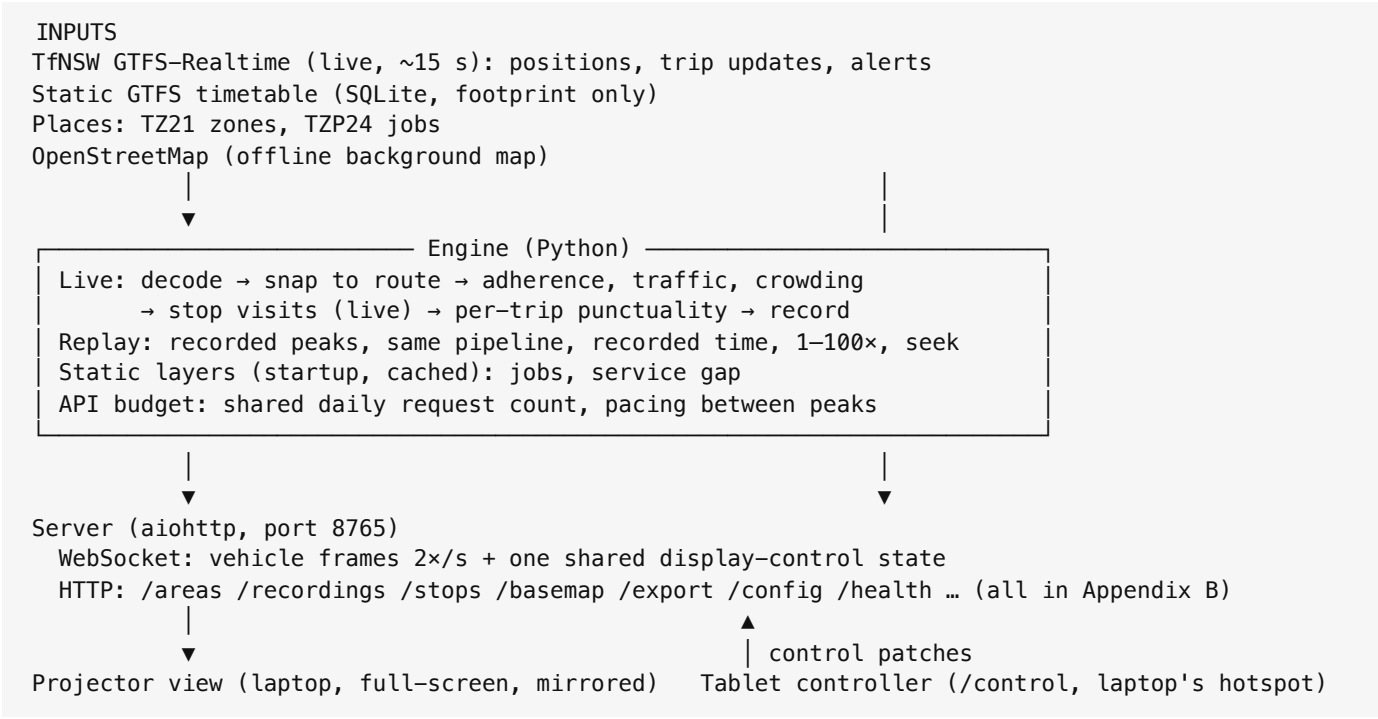
- **Server.** The Python program at the centre of the system, running on the laptop. It fetches the live data, does all the calculations, and serves the pages.
- **Engine.** The part of the server that processes data (decoding, map matching, calculations), as opposed to the part that talks to the pages.
- **Projector page** (`/`) and **tablet page** (`/control`). The two web pages the server provides. The projector page is shown full-screen on the laptop (and so on the model); the tablet page is opened on a tablet.
- **WebSocket.** A connection that stays open between a page and the server so either side can send messages at any time. The server sends vehicle positions about twice a second this way, and the tablet sends button presses back.
- **Display-control state.** One shared record, held by the server, of everything about what is shown: which layer is on, which mode is hidden, live or replay, and so on. Every page displays this one record (§4.2).
- **Replay / recording / session.** The server saves every live data download to disk (a recording). A **session** is a replayable window of recordings: an AM or PM peak on a given day, or the latest two hours.

- **launchd.** macOS's built-in service manager. Two launchd jobs keep the server running and record peaks on schedule.
- **API quota.** TfNSW limits how many requests one key may make per day — measured at about 60,000 (§5.1).

Part II — How it works

4. System architecture

4.1 The parts



There are three parts, all on one laptop:

1. **The engine** (Python, folder `engine/`). Every 15 seconds during peaks it downloads the live feeds, decodes them, works out which trip and route each vehicle is on, snaps it onto its route, classifies its punctuality, notes when it passes a stop, counts each trip once at TfNSW's measuring point, and saves the raw download to disk. At startup it also builds the slower "static" area layers — jobs and service gap — in the background, and keeps them in a cache file so later startups are almost instant.
2. **The server** (`engine/publish/server.py`, using the aiohttp library, on port 8765). It sends the vehicle positions to every open page about twice a second over a WebSocket, answers requests for layers and data (areas, stops, recordings, CSV exports and so on), and holds the display-control state.
3. **Two web pages** (`renderer/`). The **projector page** (`index.html`) draws everything on an HTML canvas: background, area colours, route corridors, stops and vehicles. The **tablet page** (`controller.html`) has the buttons, the legend and the statistics.

4.2 One shared display state

Every button on the tablet, and every key on the laptop keyboard, sends a small change (a "patch", e.g. "colour the vehicles by traffic") to the server. The server checks it is valid (`engine/publish/control.py`) and sends the

complete updated state to every open page. Pages never change what is shown on their own; they only follow the server's state.

This is why the tablet and the projection can never disagree. With two tablets open, pressing a button on one updates the other. If a page reloads, it receives the current state and shows exactly what the others show. Everything except the alignment nudge resets to safe defaults when the server restarts, so the showcase always starts from a known state. All fields are listed in Appendix A.

4.3 Live and replay go through the same code

When replaying, the engine reads recorded downloads from disk in the order they were fetched and feeds them through exactly the same processing as live data, at the chosen speed. The engine treats each download's original fetch time as "now". This means every time-based measure (the Stats tab's windows, the stop visits) runs on *recorded* time even at 25× speed, and a replay looks exactly as the live view did at the time (with today's code). While a replay plays, live polling and the server's own recording pause (to save the data quota), so **after any demonstration, switch back to Live**.

4.4 The legend reads the engine's own numbers

Since 5 November the tablet's legend does not keep its own copy of thresholds. It reads them from the server (`/config` for the punctuality limits, `/areas?meta=1` for the value range of each fifth), so if a threshold is ever changed in `config/config.yaml`, the legend changes with it.

---

5. Data sources

5.1 Live vehicle data (TfNSW GTFS-Realtime)

What it gives: vehicle positions, trip-update delays and service alerts for seven modes, refreshed about every 15 seconds; plus TfNSW's `congestion_level` and `occupancy_status` fields and per-carriage crowding for trains.

Coverage, checked live on 23 October: 164 of 269 buses reported a traffic level; crowding was reported for all buses and Metro and about half of Sydney Trains; light rail and ferries report neither field. Sydney Trains' traffic level is always "unknown" (it would be meaningless on rails). Since the teammate's fix merged on 30 October, about 70% of Sydney Trains show no usable delay (§7.2).

The quota. TfNSW doesn't document a daily limit, but on Sunday 25 October the key was refused after 59,964 requests counted from midnight and accepted again just after the next midnight. So the quota is about **60,000 requests per day, resetting at midnight Sydney time**. Polling all 15 endpoints every 15 seconds would cost about 86,000 a day — the server ran out at 7 pm that Sunday. Polling is now paced automatically (§12.3).

Quirks the engine handles (all discovered against the real feeds):

- Sydney Trains, Metro and CBD light rail use different identifiers in their live feeds than in the timetable, so their vehicles can't be joined to timetable trips by ID. They are identified by position instead: snapped to the nearest route line within 100 m.
- The ferry feed reports each vessel's current position and future scheduled ones together; only the freshest current one is kept.
- When a bus route is altered, its trip ID changes and stops matching the timetable; the engine falls back to matching on the route.

- Buses arrive as one statewide feed; only vehicles inside the footprint are kept.

5.2 Static timetable (TfNSW GTFS)

The complete GTFS bundle, filtered to the footprint and stored in SQLite (`data/static_cache/gtfs_cbd.db`). It was refreshed on 30 October because the August copy's calendar ended on 5 November, the day before the showcase. The current one runs **30 October 2026 – 28 January 2027** and has 546 routes, 2,905 stops and 3,135 route shapes in the footprint. The old copy is kept as `gtfs_cbd.2026-08-19.db`. It provides route names and colours, stop locations, route lines for map matching and drawing, and scheduled departures for the service gap.

5.3 Travel zones and jobs (TZ21, TZP24)

Zone boundaries and 2026 job estimates, taken from the team's branch (stored with Git LFS, a way of keeping large files alongside code). `scripts/prepare_demand_data.py` cuts the 109 MB statewide zone file down to the footprint. 405 zones touch the footprint. Caveat: zones differ hugely in size, so a large suburban zone can have many jobs simply because it is large. That is one reason the 150 m grid exists as an alternative (§7.6).

5.4 Opal 2016 (removed)

Tap-ons and tap-offs for 21–27 November 2016, from data.nsw.gov.au, were used for an area layer and as an input to the (also removed) pressure score until 5 November 2026, when the owner had them removed (§7.7). The download is still in `data/opal/` and in the data release, but nothing reads it.

5.5 Postcode boundaries (removed)

The ABS postcode boundaries were only used to place the 2016 bus taps, so they are no longer read either.

5.6 BOAM (removed)

TfNSW's Bus Opal Assignment Model gives, for every bus trip at every stop, estimated passengers on board (in 20-passenger bands), capacity, and scheduled and actual arrival times. 13 weekdays (28–30 September and 12–23 October) were imported and used for two street layers, a pressure input and a step 6 comparison until 5 November, when all of those were removed along with the BOAM code. The imported files are still in `data/boam/` and in the data release, but nothing reads them.

5.7 OpenStreetMap (background map)

The community-made world map, downloaded once on 4 November through the Overpass service by `scripts/fetch_basemap.py`, and stored offline in `data/basemap/basemap.json` (about 3 MB): roads, rail, water, parks, coastline and 753 place and street names. It is credited "© OpenStreetMap contributors" on the tablet.

5.8 Our own recordings

Every live download is saved, trimmed to a footprint padded by 10% (so vehicles have some history before entering the frame; untrimmed statewide feeds cost about 14 GB in a few hours). They have been recorded continuously since 24 October, at about 3.5 GB per day, 27 GB in total. Recordings from August and September were deleted: the old setup was being rate-limited and captured only 1–3 bus updates an hour. Only two weekend sessions from then (22 August, 6 September) are still usable.

5.9 Stop visits

The teammate's stop history (merged 30 October) records one row each time a vehicle passes a stop, with its crowding and delay at that moment. There were 865,907 visits by 4:27 pm on 5 November, recorded live since 30 October. Replays don't add visits (§7.13).

---

6. What it shows and how it is controlled

6.1 On the projection

Vehicles are always shown. Each is snapped to its route, moved smoothly between updates, and drawn as an arrow pointing the way it is travelling, with a short tail. Rail vehicles (trains, metro, light rail) are drawn larger than buses and ferries. Each of five groups — trains, metro, light rail, buses, ferries — can be hidden. The arrow colour follows the chosen **lens** (what the vehicle colour shows):

- **Schedule** (default): punctuality from trip-update delays (§7.2). An **Early buses** switch (tablet, under the colour choice, or **E**) turns the blue "early" colour on (default) or off; off, early buses show as on time. It only changes the live colours: the Stats tab and exports keep the official measure.
- **Traffic:** TfNSW's congestion level for the vehicle (buses only).
- **Crowding:** TfNSW's occupancy for the vehicle.

While an area layer is on, vehicles turn white so they don't compete with the area colours.

Street layers (removed 5 November). Until then the streets themselves could be coloured by TfNSW traffic, our own speed measure, crowding, typical BOAM bus load or BOAM time lost, with options for how to show a street's two directions. At the owner's request they were removed; traffic and crowding now show only as vehicle colours (*Colour the vehicles by*). How they worked is summarised in §7.5 and §7.12.

Area layers shade the model in patches. Only one is shown at a time, on either **travel zones** or a **150 m grid** (switchable, so the team can judge on the real model which reads better): Jobs and Service gap. Each shows one measure; there is no combined score.

Stops and routes (teammate's work, merged 30 Oct):

- **Stop markers** for all stops (off by default; it is a lot of dots).
- **Stop history:** each stop coloured by its typical delay (median or mean) or how often vehicles arrive crowded, over everything recorded live. A stop needs at least 20 visits to be coloured.
- **Route filter:** single out up to eight routes by name (e.g. 333, T1, L2). Their vehicles, line and stops stand out in fixed colours and everything else fades back. Each route keeps its colour when another is removed.

Background map (added 4 Nov): **Plain** (default; the model already has streets, and anything drawn would compete with them), **Detailed** (faint roads, rail, water and parks from OpenStreetMap) or **Full** (Detailed plus suburb, station, landmark, park, water and street names, placed so they never overlap). It is mainly useful on a flat screen or laptop.

Hovering (on the laptop). Pointing the mouse at a vehicle on the projector page shows a small box with its route, punctuality, exact delay, traffic and crowding (teammate's feature), and **highlights its whole route** in white (added 5 Nov). Pointing at a stop, with stop markers or stop history on, shows its name and the routes serving it, and highlights all of those routes. If a vehicle and a stop overlap, the vehicle wins. The highlight is drawn over the street colours and under the vehicles. It belongs to the laptop page only, not the shared display

state, but the projector mirrors the laptop, so it appears on the model too. Route lines come from the same `/route` lookup the route filter uses, and each stop's routes from `/stops`.

Keeping the projection clean. There are no buttons or hints on the projected page, and everything is clipped to the footprint. An optional **status line** (team panel, or **H**) shows the mode, clock and replay state. It is currently the only way the projection itself says "this is a replay", so either turn it on during replays or say so out loud.

6.2 Time: live and replay

- The system starts in **Live**.
- **Replayable sessions:** every recorded AM peak (7:00–9:00) and PM peak (16:30–18:30) that has at least 10% of the expected bus updates, plus "Latest 2 hours". The 10% minimum hides sessions so sparse they would replay as an almost empty model.
- **Choosing one (since 5 Nov).** The tablet's Time section has three buttons of the same style: **Live now**, **Latest 2 hours** and **Replay mode**. Replay mode opens a **day dropdown** listing every recorded day, newest first (14 days, 25 peaks today), next to **AM peak** and **PM peak** buttons. Opening it changes nothing on the model until a peak is tapped. A missing peak's button is greyed out, and a note flags peaks under 90% recorded (e.g. "partly recorded: AM 66%, PM 66%" for the weekend of 31 Oct–1 Nov). Changing the day while a peak is playing jumps to the same peak on the new day. Only one Time button is lit at a time. Before 5 November the tablet showed only the first seven sessions (the latest plus three days), even though the server listed all of them.
- **Speed:** 1× (real time), 10×, 25× (default; a two-hour peak in under five minutes) or 50× on the tablet; the server accepts 1–100×. Replays loop.
- **Pause and play** (tablet, or Space): the projection freezes and resumes without jumps.
- **Scrubbing:** tap anywhere on the timeline, use the –5/+5 minute buttons, or ←/→ for one minute. On a jump, the engine first processes the 10 recorded minutes before the target as fast as it can, so the street layers are already filled when playback resumes.
- **Remember:** live polling and recording pause during any replay. Switch back to Live afterwards.

6.3 The tablet (`/control`)

The tablet has three tabs.

Display (for visitors) holds:

- time and replay controls, and at the bottom of the Time section, **Download a recorded peak (CSV)** (added 5 Nov): pick a day and AM or PM peak, then download *Vehicles*, *Streets*, *Stop visits* or *Trips on time*. The first export of a peak takes about 2 minutes, after which it downloads straight away (§12.9, Appendix D);
- which vehicle groups are shown, and the dot colour (lens);
- stop markers and stop history (median or mean);
- the route filter;
- the area layer and area shape (zones or grid);
- the background map;
- **What the colours mean**, a legend for whatever is currently on.

Since 5 November the legend gives every colour a definition with its cut-off, not just a name. For example:

- "On time: up to 5 min late: Sydney Trains (TfNSW punctuality standard). Up to 5 min 59 s late and no more than 59 s early: buses (Greater Sydney Bus Contract, 2023)…"

- "Jobs, second fifth: 14–28 jobs per 150 m square."

Where TfNSW only provides categories (its traffic and crowding fields), the legend says so and names the feed value behind each colour rather than inventing ranges. It also explains the stop-history bands. Every definition is in §8.

Team (open to anyone since 5 November, when its PIN was removed) holds:

- the theme (dark or light);
- the verification overlay (route corridors drawn bright, to check alignment);
- the status line on the projection;
- alignment: the nudge (shift by 2 px or 20 px, and scale), **corner fit** (drag the footprint's four corners onto the model's; teammate's, 5 Nov) and crosshair calibration;
- system health: vehicles per mode, how old each feed is, API requests used today, and the current polling interval.

Stats shows the share of trips on time by mode over the last 15 minutes, hour and 2 hours: each trip counted once, at TfNSW's own measuring point (buses leaving their first stop, other modes at their final stop, §7.2), with the number of trips. It also shows active service alerts affecting the footprint, and the busiest interchanges.

6.4 Laptop keyboard (backup)

Every key goes through the server, so the tablet stays in sync:

C calibrate · F corner fit · V verification overlay · H status line · T theme · M background map · S stop markers · O stop history · L dot colour · E early buses on/off · A area layer · G zones/grid · Space pause/play · 1–5 mode groups · arrow keys nudge (Shift ×10) · [ ] scale · 0 reset nudge.

---

7. How every number is calculated

This section explains each calculation in enough detail to reproduce it, with the reason behind each choice and a worked example. File names point to where the code lives.

7.1 Putting vehicles on their routes (`engine/normalise/map_match.py`, `enrich.py`)

The problem. A live feed gives each vehicle a GPS position and, usually, a trip ID. GPS positions drift several metres off the road, and some modes' trip IDs don't match the timetable at all.

What the engine does, for every vehicle on every update:

1. **Identify the trip and route.** If the live trip ID is in the timetable, use it. If not, but the route ID is, use the route. If neither matches, find the nearest route line within 100 m and use that. This last step is what places Sydney Trains, Metro and CBD light rail, whose live IDs use a different naming scheme from the timetable. Each vehicle records which of these worked, so nothing is silently guessed.
2. **Snap to the line.** Move the position to the nearest point on the trip's route line, and record how far along the line that point is (its "distance along shape"). Drawing uses the snapped position, so vehicles sit on the street; the distance along the line is what our speed measure (§7.10) and the street segments (§7.5) use.

Why it was rebuilt for speed. At first the search compared every vehicle with every whole route line. Almost every route line in the timetable passes through the CBD, so the search could rule almost none out: it took

about 14 ms per vehicle and 92% of the server's processor time. The search index now holds 32-point pieces of each line, so most are ruled out instantly. The live server went from about 85% processor use to a few percent, and replays from about 11× to about 96× real time.

7.2 Punctuality (`engine/normalise/adherence.py`, `trip_updates.py`)

The measure. A vehicle's delay is TfNSW's predicted delay, in seconds, at the vehicle's **next stop**, taken from the trip-update feed. Comparing with the next stop (not the trip's first stop, and not the vehicle's position against the timetable) means the colour describes how late the vehicle is *now*, where it is.

The on-time limits, and where each comes from (changed 5 November; set in `config/config.yaml`, `punctuality`). Until then the limits were defaults from the original brief, and "very late" meant twice the limit, neither with a source. Now each limit is taken from a published definition where one exists:

Mode	On time if	Source
Sydney Trains	no more than 5 min late	TfNSW's punctuality standard: a train is punctual if it reaches its final destination within 5 minutes of the timetable (target: 92% of peak services)
NSW TrainLink intercity	no more than 6 min late	The same TfNSW standard, 6 minutes for intercity
Buses	no more than **59 s early** and no more than **5 min 59 s late**	Greater Sydney Bus Contract, Schedule 4 (Key Performance Indicators), 2023, measured at the first stop (target 95%). The older Sydney Metropolitan contract allowed up to 1 min 59 s early
Metro, light rail, ferries	no more than 5 min late	**No published definition found.** TfNSW's Sydney Trains 5-minute standard is applied by the project, and the legend says so

The states:

State	Rule	Colour
Early	buses only: more than 59 s early	blue
On time	within the mode's limits	green
Late	later than the mode's limit	amber
Missed its slot ("very late")	later than the limit **and** later than the scheduled gap to the next trip on the same route and direction	red
Cancelled	TfNSW marks the trip cancelled	red
Unknown ("no schedule data")	no usable prediction	grey

Why buses can be early and trains can't. The bus contract counts leaving more than 59 seconds early as a failure, because passengers arriving on time miss the bus. TfNSW's rail standard only sets a lateness limit, so a train running early still counts as punctual.

"Missed its slot" replaces the old 2× rule. TfNSW has no "very late" category, so the project defines one that can be checked against the timetable rather than picked: a vehicle is red when it is past its on-time limit and also later than the scheduled gap to the next trip on its route and direction (`engine/normalise/headways.py`). At that point a passenger would have done better catching the next service. The gap is worked out once at startup from the timetable: for each trip, the time from its first departure to the next trip's on a typical Tuesday, Saturday and Sunday. It means more for a bus every 5 minutes than for one every 30. Transport for London measures frequent buses in a related way, by the extra time passengers wait rather than by lateness. Limitation: Sydney

Trains, Metro and CBD light rail can't be matched to timetabled trips (their live trip IDs differ), so they never show red for this reason; past the limit they are just Late.

Worked examples:

- A bus predicted 7 minutes late at its next stop is past the 5 min 59 s limit. On a route every 20 minutes it is **Late** (amber). On a route every 5 minutes it has **missed its slot** (red).
- A bus 90 seconds early is **Early** (blue).
- A Metro train 3 minutes late is **On time**. Under the old 2-minute default it showed as late.

Live colour versus TfNSW's official figures. The colour is a *live* reading wherever the vehicle is. TfNSW's official figures are measured at fixed points: trains at their final destination, buses as they leave their first stop. So the tablet's Stats tab (§6.3) counts **each trip once at those points** (`TripPunctuality` in `engine/normalise/aggregate.py`):

- **Buses** are counted by their delay as they leave their trip's first stop, from the stop-visit tracker.
- **Every other mode** is counted by TfNSW's predicted delay at the trip's final stop. The latest prediction wins, which is the one made closest to arrival. Many trains finish outside the model's area, so their figure is a prediction, not an observed arrival.
- **Not counted:** trips with no live prediction, and buses whose first stop is outside the model's area, since they are never seen leaving it.
- **Cancelled trips** count as not on time.

On the Wednesday 4 November AM peak, measured this way, Sydney Trains were **94% on time** (287 trips; TfNSW's target is 92%), buses **93%** (686 trips; contract target 95%), CBD light rail 80%, Inner West light rail 89%, ferries 97% and Metro 100%.

Something the new rules show. On that same peak, **26% of live bus updates were Early**, but **no bus was early leaving its first stop**. Mid-route, TfNSW predicts many buses ahead of their timetabled times, which fits the timetable padding found in step 6 (§10.1). The contract only checks earliness at the first stop and timing points, so mid-route earliness doesn't count against operators. To a passenger at that stop, though, an early bus is still a missed bus. So the owner chose to keep it, with an **Early buses** switch on the tablet (on by default) to hide it when it's a distraction: off, early buses are drawn green. The Stats tab and exports are unaffected.

The placeholder-zero fix (teammate's, merged 30 Oct). Some feeds send a delay of exactly 0 seconds for every stop of a trip when they have no real prediction. Treating that as "on time" made almost every Sydney Train green. Now an exact 0 s across a whole trip counts as "no prediction", so about 60–70% of Sydney Trains show grey. The same fix makes Metro, light rail, ferries and NSW TrainLink read the next stop's delay rather than the trip's first stop's, which was always 0.

7.3 Interchange activity (`engine/normalise/aggregate.py`)

For each of nine interchanges (Central, Town Hall, Wynyard, Martin Place, Circular Quay, Barangaroo, Gadigal, Museum, St James), the engine counts the **distinct** vehicles seen within **300 m** in the last **5 minutes**. "Distinct" matters: a train waiting at a platform across several updates counts once, not once per update. The counts feed the "busiest interchanges" bars on the tablet's Stats tab.

Until 5 November they also drove glowing rings on the projection that grew and brightened with activity (the "interchange pulses"). The owner had them removed: they are no longer drawn, and their toggle and **P** key are gone.

7.4 TfNSW's traffic and crowding fields (`engine/normalise/levels.py`)

TfNSW's two "how is it going" fields are categories. The project turns each category into a level from 0 to 3 so readings from several vehicles can be averaged:

Level	Traffic (`congestion_level`)	Crowding (`occupancy_status`)
0	RUNNING_SMOOTHLY	EMPTY, MANY_SEATS_AVAILABLE
1	STOP_AND_GO	FEW_SEATS_AVAILABLE
2	CONGESTION	STANDING_ROOM_ONLY
3	SEVERE_CONGESTION	CRUSHED_STANDING_ROOM_ONLY, FULL, NOT_ACCEPTING_PASSENGERS
none	UNKNOWN or missing	missing

Some notes on these fields:

- **Seven crowding categories become four.** EMPTY is folded into "many seats" and the three "no room" states into one, because seven colours can't be told apart on a projection.
- **Traffic counts buses only.** Metro reports a level, but it isn't road traffic. Sydney Trains' level is always unknown.
- **Trains without a vehicle-level crowding value** use the **middle carriage** of the per-carriage readings.
- **There are no numbers behind these categories.** TfNSW doesn't publish how its traffic levels are decided (from speed, delay or something else) or how many passengers each crowding category means. So no numeric range can honestly be given for them. That is exactly why the project built its own congestion measure with stated cut-offs (§7.10).

7.5 Street segments and pooling (removed 5 November)

Until 5 November, street layers coloured each route's line in 200 m pieces. Each piece showed the mean level of the vehicles seen on it in the last 10 minutes. On 4 November this was improved: readings were pooled along shared streets, so on a street served by many routes one bus no longer stood for the whole street. Streets resting on one vehicle were drawn faint, and the two directions could be shown together, side by side or one at a time. The street layers and this code (`engine/normalise/segments.py`) were removed at the owner's request. The code is still in git history (commit `5676e3b` and earlier).

7.6 Jobs, departures and the service gap (`engine/demand/areas.py`)

These layers use two kinds of area:

- **Travel zones** (405 in the footprint);
- a **150 m grid**, whose squares are all the same size, so large zones don't stand out just for being large.

Grid squares take an area-weighted share of the zones they overlap. For example, a square lying half in a zone with 1,000 jobs, where that half is 5% of the zone's area, gets 50 jobs from it.

Jobs: the zone's TZP24 2026 job estimate.

Departures: a measure of how much public transport a worker in the area can reach. It counts scheduled departures, of any mode, from stops within **200 m** of the area, between **7:00 and 9:00** on a reference weekday (**Tuesday 10 November 2026**, inside the current timetable). A stop counts for every area within 200 m of it, and a trip counts once for each stop it serves there. So it measures service you could walk to, not the number of distinct trips.

Service gap: the area's percentile rank for jobs minus its percentile rank for departures, among areas with any jobs.

- It ranges from –1 to +1.
- **Positive** means more jobs than service compared with the rest of the footprint (relatively under-served).
- **Negative** means more service than jobs.

Worked example: an area with more jobs than 90% of areas (rank 0.9) but more departures than only 30% (rank 0.3) has a gap of +0.6. That is "much more jobs than service".

Why a rank difference and not a ratio. Jobs divided by departures explodes when departures are near zero, and comparing the two raw numbers would mix units. Ranks put both on the same 0–1 scale with no units.

Bands:

Gap	Label
–0.40 or less	much more service than jobs
–0.40 to –0.15	more service
–0.15 to +0.15	balanced
+0.15 to +0.40	more jobs
over +0.40	much more jobs than service

Fifths (jobs layer). Areas with any value are ranked and split into five equal-sized groups. Areas with none aren't coloured. The actual value range of each fifth is in §8.3. The ranges differ between zones and the grid, and the tablet shows the ones for whichever shape is on.

7.7 Opal 2016 (removed 5 November)

Until 5 November an area layer showed the average weekday's tap-ons plus tap-offs between 7:00 and 9:00 for 21–25 November 2016, and the same taps were one of the pressure score's inputs (the score itself was removed later the same day, §7.8). At the owner's request both were removed: the layer, its tablet button and legend, its pressure weight, and the code that placed station and bus taps on the map. Reasons it was the weakest input: it was ten years old, predated the Metro and the CBD light rail, and placed bus taps only by postcode. The section number is kept so references elsewhere stay valid.

7.8 Pressure (removed 5 November)

Until 5 November a *Pressure (combined)* area layer gave each area one score from up to five inputs (traffic now, crowding now, service gap, Opal 2016, typical BOAM load). Each input was a percentile rank across areas, missing inputs counted as a neutral 0.5, inputs were averaged with weights of 0–3 set on the team panel, and areas were coloured by fifths of the score. At the owner's request the layer, its live scoring on the server (`/pressure`), the weight sliders and the weights setting were removed, so every area layer now shows one measure on its own. The section number is kept so references elsewhere stay valid.

7.9 BOAM bus sections (removed 5 November)

Until 5 November, BOAM data was turned into two values for each stretch between consecutive bus stops, for each weekday peak. Both were averaged over 13 weekdays:

- **Typical load:** passengers on board leaving the stop, divided by the bus's capacity.
- **Time lost per 100 m:** how much further behind timetable buses fell on that stretch.

They coloured two street layers, fed the pressure score, and were compared with our congestion measure in step 6 (§10). All of these, and the BOAM code (`engine/demand/boam.py`), were removed at the owner's request. The code is still in git history.

7.10 Our own congestion measure: the travel-time index (removed 5 November)

This measure coloured the "Traffic (our measure)" street layer, and was the subject of the step 6 analysis. Its code (`engine/normalise/travel_time.py`) and the analysis scripts were removed with the street layers on 5 November, and remain in git history. The method is kept here because §10's results depend on it.

Why build one. TfNSW's traffic label is published without a method (§7.4). A measure we can explain lets the project say exactly what a colour means, and test whether TfNSW's label behaves like a measure of slow streets (§10).

The reading.

- For each bus placed on its route line, the engine waits until the bus has moved **at least 300 m** along the line since the last reading.
- It then computes the bus's average speed over that stretch. Stops and traffic lights are included on purpose, because the free-flow speed below includes them too, so they cancel out.
- The reading is attributed to the 200 m piece at the stretch's midpoint.

The free-flow speed. This is a street's "empty" speed: the **85th percentile** of the same kind of reading taken **at night (22:00–05:00)** from our own recordings, for each 200 m piece.

- Pieces with too few night readings fall back to the whole route's night speed.
- The current baseline (`scripts/build_freeflow.py`) has 4,712 pieces from 7 nights, plus 217 whole-route fallbacks.
- Pieces with no baseline at all show grey.

The index. TTI = free-flow speed ÷ current speed, capped at 10.

- *Example:* a street buses cover at 30 km/h at night, now crossed at 12 km/h, gives TTI 30 ÷ 12 = 2.5. The trip takes two and a half times as long as at night.

Levels on the model (until 5 November), chosen to be easy to explain to visitors:

TTI	Level
up to 1.5	running smoothly
1.5–2	stop and go
2–3	congested
over 3	severely congested

For the statistical comparison with TfNSW, different cut-offs are used (2.66 / 2.98 / 3.32). These were fitted so our four levels occur as often as TfNSW's do. Otherwise the comparison would mainly measure a difference in how often each says "congested" (§10).

Why a night baseline, and not the timetable? Comparing with the timetable was tried first and failed: timetables are padded, so buses "gained" time even in severe congestion (§10.1).

7.11 Street colours from BOAM and the live layers (removed 5 November)

Both kinds of street layer were removed (§7.5, §7.9).

7.12 How street lines were drawn (removed 5 November)

Until 5 November, each street was one solid 4.5 px line; its soft glow had been removed on 4 November. Lines were laid down best evidence first, so no colour peeked out from under another. The drawing code went with the street layers.

7.13 Stop history (`engine/history/`, teammate's work)

What a visit is. A visit is recorded when a trip's "next stop" changes, meaning the vehicle has just passed that stop. The reading kept is the **last** crowding and delay seen while that stop was still next: what the vehicle looked like as it left. This uses TfNSW's own next-stop value (present on 97.4% of trip updates, checked live), rather than guessing from distance to the stop.

What is computed per stop, over everything recorded:

- median and mean delay;
- the share of arrivals at crowding level 1 or more ("few seats" or fuller);
- delay split by whether vehicles arrived seated or crowded. Each side needs at least 5 visits, otherwise no figure is given rather than one based on two readings.

Delays outside –30 minutes to +3 hours are treated as glitches and ignored.

Median versus mean. Both are offered because they say different things. The median is the typical vehicle; the mean also carries how bad the occasional very late one is. A big gap between them marks an unreliable stop.

What a stop's colour does and doesn't say. A vehicle arrives at a stop carrying all the delay it picked up earlier in its trip. A red stop therefore means "vehicles are typically late by the time they get here", not "this stop causes delay". That is also why the CBD tends to look greener: it has more rail, and buses pick up more delay towards the ends of their routes. To find where delay is *added*, compare consecutive stops, or use the BOAM "Where buses lose time" layer (§7.9).

A finding from the teammate's notes (as of 4 November). At 610 of 662 stops (92%) with enough visits, crowded vehicles arrived later than seated ones. That is an association, not proof that crowding causes delay: crowded vehicles are likely also the ones running at peak times and on the busiest routes, which are late for other reasons too.

Only live data is recorded (fixed 30 October). The visit recorder sat in code that replays also run through, so every replay, and every loop of a replay, wrote recorded visits into the history again. Now visits are written only while live. Checked: 30 seconds of a 50× replay added no visits.

Speed. The `/stop_history` address recomputes everything at most once a minute, without holding up anything else on the server. A full recompute takes about 2 seconds at 866,000 visits (5 November). A cached answer takes 0.03 seconds.

7.14 Background map (`scripts/fetch_basemap.py`)

Downloading. The map data comes from OpenStreetMap, through its public Overpass service. The public servers failed several times (errors 502 and 504), so the script retries, falls back to a second server, and keeps the raw download so it can be rebuilt without downloading again (`--cached`).

Drawing. The map is drawn as lines and shapes through the same projection as everything else, so it lines up with the model after calibration. Picture tiles, the usual way web maps work, would have needed warping to match.

What each option shows:

- **Plain** (the default) draws no map at all.
- **Detailed** draws major and minor roads, rail, water, parks and coastline. These are kept faint so data stays on top, and hidden while the verification overlay is on. Two problems found while checking it:
    - Road tunnels are left out, because drawing the Harbour Tunnel on the surface would mislead on a physical model.
    - Each water and park outline is filled separately. Filled together, overlapping outlines cancelled each other out and the harbour vanished.
- **Full** adds 753 labels. In order of importance: suburbs, water, stations and wharves, landmarks, parks over 4 hectares, neighbourhoods, then street names.
    - There is one label per wharf ("Circular Quay", not one per berth).
    - Street names are angled along the road and repeated about every 1 km.
    - Labels are placed greedily in that order. Any label that would overlap one already placed, or wouldn't fit wholly on the model, is skipped.
    - Bus interchanges and railway junctions aren't labelled, because they crowded out stations and places.
    - Labels are drawn above the street colours and below the vehicles.
    - At projector resolution the smallest labels (10 px street names) may be too small to read, so Full is probably most useful on a screen.

---

8. Every colour, defined

This is what the tablet's *What the colours mean* section states (since 5 November), in one place. **Absolute** bands mean the same thing on any day. **Relative** bands rank areas against each other, so they say "top fifth of the footprint", not a fixed amount. **Categorical** bands are TfNSW's own categories, which have no published numbers behind them.

8.1 Vehicles (the lens)

Lens	Colour	Definition
On schedule?	Blue — early	Buses more than 59 s early (Greater Sydney Bus Contract). Other modes are never early
	Green — on time	No more than 5 min late (Sydney Trains, TfNSW standard), 6 min (intercity, TfNSW), 5 min 59 s late and 59 s early (buses, bus contract), 5 min (metro, light rail, ferries: TfNSW's rail standard, applied by the project)
	Amber — late	Later than that
	Red — missed its slot / cancelled	Later than the limit and later than the scheduled gap to the next service on its route; or cancelled
	Grey — no schedule data	No live delay prediction for the vehicle's trip
Traffic (TfNSW)	Green / yellow / orange / red	The bus's `congestion_level`: RUNNING_SMOOTHLY / STOP_AND_GO / CONGESTION / SEVERE_CONGESTION. TfNSW publishes no numbers behind these

Lens	Colour	Definition
	Grey	UNKNOWN or missing. Always the case for trains and metro, which aren't in road traffic
Crowding (TfNSW)	Blue / purple / pink / red	EMPTY or MANY_SEATS_AVAILABLE / FEW_SEATS_AVAILABLE / STANDING_ROOM_ONLY / CRUSHED_STANDING_ROOM_ONLY, FULL or NOT_ACCEPTING_PASSENGERS. These are categories, not passenger counts. Trains use the middle carriage
	Grey	No occupancy sent

8.2 Streets (removed 5 November)

The street layers and their colours were removed; traffic and crowding are shown only on the vehicles (§8.1).

8.3 Areas

These are the actual ranges as built on 5 November. The tablet reads them live from `/areas?meta=1`. The bottom fifth reads "up to", because small fractions of a job round down to 0.

Layer	Lowest fifth	2nd	Middle	4th	Highest fifth
Jobs per travel zone	up to 391	393–726	727–1,260	1,263–2,983	3,065–21,409
Jobs per 150 m square	up to 14	14–28	28–54	54–128	128–11,761

Both are relative. Note how uneven the top fifth is: it runs from about 3,000 to over 21,000 jobs per zone. The busiest places, such as the CBD core and the big stations, are far above everything else, which is why fifths are used rather than evenly spaced bands.

- **Service gap** (fixed bands on the rank difference, §7.6): –0.40 or less, much more service than jobs; –0.40 to –0.15, more service; –0.15 to +0.15, balanced; +0.15 to +0.40, more jobs; over +0.40, much more jobs than service.

8.4 Stops and background

Overlay	Bands
Stop history, delay (median or mean on arrival)	Under 1 min, 1 to under 2 min, 2 to under 4 min, 4 min or more late. Early counts as under 1 min
Stop history, when the crowding lens is on	Share of arrivals with few seats or fuller: under 10%, 10 to under 25%, 25 to under 50%, 50% or more
Stop history, any	Stops with fewer than 20 visits aren't coloured
Background map	Not data: roads, rail, water and parks, drawn faint

---

Part III — Why it is built this way

9. Design decisions

9.1 Why the teammates' calculations weren't reused

The teammates' dashboard set the project's direction, but its congestion score had flaws that would have made the model misleading:

Flaw in the dashboard's congestion score	Why it misleads
A bus free-flow speed (30 km/h) applied to trains and Metro	Rail isn't in road traffic, so "congestion" on rail meant nothing
Absolute delay treated as local	A bus that left Parramatta late reads as congested in the CBD, where it may be moving freely
Instantaneous speed	A bus stopped at a stop or red light reads as slow
Bunching that ignores the timetable	High-frequency routes run close together by design
Grid cells averaging whatever is in them at that instant	No history, and results depend on arbitrary cell boundaries

Its "demand" was jobs per zone. That describes land use, not transit demand, so here it is labelled **jobs** and used inside the service gap. The dashboard also had a mapping-service API key committed in plain text.

9.2 The major decisions, explained

The display state lives on the server and is broadcast.

- *Why:* the projector mirrors the laptop, so controls must be elsewhere, and two places (tablet and keyboard) can change the display. With one copy on the server, every page shows the same thing, and a reloaded page catches up instantly.
- *Rejected:* each page keeping its own state, which drifts out of step.

A tablet controller, the keyboard as backup, a separate team panel, the laptop's hotspot.

- *Why:* visitors need buttons, and none can appear on the projection.
- *Rejected:* visitors' own phones, since they would need the network details and university Wi-Fi often blocks devices from reaching each other.
- The team panel had a PIN until 5 November; the owner had it removed, so anyone at the tablet can open alignment and calibration.

A plain background by default, clipped to the footprint; an optional offline map.

- *Why:* the model already has streets, and a drawn map would compete with them, while anything outside the footprint would land on the table.
- The optional map (4 Nov) helps on a screen. It is stored offline so it works without internet, and drawn through the calibration so it lines up.
- *Rejected:* live map tiles, which would need warping to match the projection.

Street layers: built, improved, then removed (5 November).

- *History:* the streets were coloured on 200 m route pieces. On 4 November they were pooled along shared streets, drawn faint when thin, and given direction modes, after measurements showed one bus often standing for a whole street (§7.5).
- *Removed* on 5 November at the owner's request, together with their engine code, BOAM and the step 6 analysis code. Traffic and crowding now show only on the vehicles.

Zones and grid both available.

- *Why:* which reads better on a physical model can only be judged on the model. Zones match TfNSW's own data; the grid removes the effect of zone size.

Service gap as a difference of ranks.

- *Why:* it has no units, and a jobs-to-departures ratio explodes where departures are near zero.

No combined score (5 November).

- *Why:* the owner wanted each area measure kept independent of the others. A pressure score that averaged ranks of traffic, crowding, service gap and bus load had been built (§7.8), and was removed.

Replay through the same engine, on recorded time.

- *Why:* one code path means a replay is exactly what live showed, and the 10-minute windows stay correct at any speed.
- *Rejected:* pre-rendered videos, which couldn't change layers during the demo.

A 25× time-lapse as the showcase default.

- *Why:* it shows congestion building and clearing within minutes, works at any time of day, and doesn't depend on venue Wi-Fi.

The replay picker: Replay mode, then a day dropdown, then AM/PM.

- *Why:* 25 peaks, growing by two each weekday, would fill the tablet as buttons. Opening the picker doesn't change the model, so browsing days is safe.

The legend reads the engine's thresholds, and doesn't invent ranges for TfNSW's categories.

- *Why:* a copy of the numbers in the legend would drift from the code. Made-up speed or passenger ranges for TfNSW's categories would mislead.

Recordings trimmed to the footprint.

- *Why:* statewide feeds cost about 14 GB in a few hours, and replays only need what can appear on the model.

A shared daily request budget, full speed only in peaks.

- *Why:* the quota is about 60,000 requests a day, measured, and constant 15-second polling ran out at 7 pm.
- *Rejected:* fixed 15-second polling.

Network errors retried within 30 seconds.

- *Why:* a Wi-Fi drop should cost half a minute, not the five-minute back-off used for "too many requests" errors.

The timetable refreshed on 30 October.

- *Why:* the old calendar ended 5 November, the day before the showcase, and every vehicle would have lost its schedule.

The server kept alive by launchd (3 November).

- *Why:* the Mac shut down on 3 November, the hand-started server stayed down for seven hours, and a PM peak was lost.

---

10. Step 6: our congestion measure compared with TfNSW's

The analysis code (`engine/normalise/travel_time.py`, `engine/analysis/collect.py`, `scripts/analyse_congestion.py`, `build_freeflow.py`, the two experiment scripts) was removed on 5 November with the street layers, at the owner's request. The results below come from the runs on 30 October. They can't be re-run from the current code, but the code is still in git history (commit `5676e3b` and earlier), and the outputs are still in `data/analysis/`.

10.1 First attempt: delay against the timetable (dropped)

The plan was to measure how much time buses lose against the timetable per 100 m of street. On the 28 October AM peak the median bus *gained* 3–5 seconds per 100 m — even on streets where TfNSW reported severe congestion.

The reason is that bus timetables are padded. Buses are scheduled generously and wait at "timing points" when early, so time against the timetable mostly measured the padding, not the traffic. Variants (stretches of 100, 300 and 500 m, with stops included or excluded) didn't fix it. This is worth a paragraph in the final report as an informative negative result.

10.2 Adopted method: travel-time index

The method finally used is the travel-time index against night-time free flow, described in §7.10. It needs no timetable at all.

10.3 How agreement was measured

Every recorded weekday peak was replayed through the engine. Each of our readings (a bus's 300 m+ stretch) was paired with the `congestion_level` the same bus reported at the end of that stretch. Agreement was then measured three ways:

- **Per reading:** do the two give the same level for the same bus at the same moment? Measured with exact agreement, agreement within one level, and quadratically weighted κ (§3.3), which counts a one-level miss as mostly agreeing and a three-level miss as not at all.
- **Per segment per 10 minutes:** the same comparison after averaging both within each street piece and 10-minute window.
- **Per street over a whole peak:** do streets that TfNSW calls more congested also have higher TTI on average? Measured with Spearman's ρ (§3.3).

10.4 Experiment: which baseline and stretch length agree best?

Run by `scripts/experiment_congestion_collect.py` and `_evaluate.py`. To avoid fitting and scoring on the same data, the thresholds were fitted on Monday–Wednesday peaks and scored on Thursday–Friday.

Baseline	Stretch	Has a baseline	Fitted thresholds	κ per reading	κ per 10 min	Spearman per street
Night	300 m	62%	2.66 / 2.98 / 3.32	0.009	0.015	**0.275**
Night	600 m	61%	2.42 / 2.70 / 3.01	0.003	0.012	0.104
Night	1000 m	62%	2.30 / 2.59 / 2.89	–0.009	0.003	0.181
Midday	300 m	73%	1.91 / 2.12 / 2.34	–0.022	–0.018	0.182
Midday	600 m	73%	1.70 / 1.87 / 2.05	–0.025	–0.016	0.038

Baseline	Stretch	Has a baseline	Fitted thresholds	κ per reading	κ per 10 min	Spearman per street
Midday	1000 m	74%	1.62 / 1.80 / 2.01	–0.023	–0.013	0.156

The night baseline with 300 m stretches did best, so it is the one used.

10.5 Final result

Run by `scripts/analyse_congestion.py` over 9 weekday peaks: 190,251 bus stretches, of which 62% had a baseline. The analysis thresholds were used.

Scale	Result
Per reading (97,488 pairs)	Exact agreement 63%, within one level 73%, **weighted κ 0.01**
Per segment per 10 min (83,634)	Exact agreement 63%, κ 0.016
Per street, whole peak (1,184 streets)	**Spearman ρ 0.24**
Median TTI by TfNSW level	Smooth 1.75 · Stop and go 2.03 · Congested 2.02 · Severe 1.93
Against BOAM time lost (2,207 segments)	Spearman 0.08

10.6 What it means

Per bus or per 10 minutes, the two measures don't agree beyond chance. The 63% "exact agreement" sounds good, but both measures call about 77% of readings "smooth", so they match by coincidence most of the time. κ removes that coincidence and leaves almost nothing.

Averaged per street over a peak, there is a weak but real relationship (ρ ≈ 0.24–0.28). Streets TfNSW calls congested do tend to be somewhat slower.

The median TTI barely changes across TfNSW's levels (1.75 for smooth, about 2 for the others). Buses TfNSW labels "severe" aren't, on average, moving slower than those labelled "stop and go".

Likely reasons:

1. The night baseline has fewer passengers boarding, so every peak street reads somewhat slower.
2. A single bus's 300 m reading is noisy: one long stop changes it a lot.
3. TfNSW's label is undocumented and per vehicle. It may not be a measure of the street that bus just drove at all.
4. BOAM time lost is measured against the timetable (so it is affected by padding), and three of its days are school holidays.

Conclusion for the final report: the two capture different things and only line up when averaged per street over a peak. Until 5 November the model showed both, labelled as such; since then only TfNSW's label is shown (on the buses).

This analysis ran before street pooling (4 November), on per-piece readings.

Outputs are in `data/analysis/`:

- `congestion_agreement.json`;
- `congestion_disagreements.geojson` (337 segments where the two differ by 2 or more levels);
- `experiment_results.json`;
- `freeflow.json`.

---

11. Testing, validation and performance

11.1 Automated tests

There are 173 automated tests (`pytest`), which run in about 2 seconds without internet access. They cover:

- the timetable filter and map matching;
- punctuality, interchange counts and statistics;
- calibration maths and footprint trimming;
- the display-state rules (every allowed and rejected value, including the corner-fit offsets and their saving across restarts);
- replay: pacing, looping, coverage, jumping, pausing, choosing between two recorders' files;
- area maths, including the per-fifth ranges;
- the request budget, the per-trip punctuality count and the timetable gaps;
- the teammate's stop tracker, history, trip-update and route-focus code;
- the background map build: layers, labels, tunnels, multi-part water outlines.

What the tests don't cover: the projector and tablet pages' JavaScript. Those are checked by eye in Chrome after every change.

11.2 Live checks

- TfNSW field coverage on 23 October: §5.1.
- After the 30 October timetable refresh, all 324 live vehicles matched a route name.

11.3 Visual checks in Chrome

Checked by eye:

- the projector on every layer;
- the background map in Plain, Detailed and Full, in both themes;
- the tablet's three tabs;
- the replay picker: a full day, a 66% day, days missing a peak, and the open and closed states;
- every legend combination: each lens, each area layer on zones and grid, stop history.

Problems found and fixed this way:

- lines spilling past the model's edge;
- a hidden row showing;
- the recordings list missing after startup;
- the nudge buttons' layout;
- the harbour vanishing from the map;
- a clipped tablet button;
- "TfNSW" lower-cased in a heading;
- a raw date (20261110) in the service-gap note;
- "Live now" staying lit while Replay mode was open.

11.4 Performance

Measure	Before	After
Live server processor use	~85%	1–18%, averaging ~7%, at the 5 Nov PM peak with 487 vehicles
Replay throughput, dense data	~11× real time	~96×
50× replay, measured	—	50.0×
Static layers: build / from cache	—	~5 s / 0.02 s
Daily requests	Quota ran out at 7 pm	~57,000 a day, peaks at full rate
`/stop_history` full recompute	~13 s at 586k visits (3 Nov)	~2 s at 866k visits (5 Nov); 0.03 s cached

---

Part IV — Running it

12. Operating it

12.1 Starting and stopping

The server runs as a launchd job (`com.pulsecbd.server`) installed on 3 November. It starts when the owner logs in, and restarts within 30 seconds whenever it stops. It runs under `caffeinate`, which stops the Mac idling to sleep while it runs.

```
python scripts/install_server_service.py                 # install the job and start the server
launchctl kickstart -k gui/$(id -u)/com.pulsecbd.server  # restart, e.g. after a code change (~30 s down)
python scripts/install_server_service.py --uninstall     # stop for good (a plain `kill` only restarts it)
```

- Because it is a per-user job, after a reboot someone must log in before the server starts.
- Startup takes about 15 seconds. The area layers finish building in the background a few seconds later, and their buttons are greyed out until then.
- **Projector:** open `http://127.0.0.1:8765/` full-screen on the laptop.
- **Tablet:** open `http://<laptop address>:8765/control`; the address is printed in the log.
- After any code change, reload both pages.
- For a one-off run in a terminal instead, uninstall the job first (otherwise the port is taken), then run `python scripts/run_publish.py`.

12.2 Recording

The server records everything it downloads while live (files tagged `serve`).

A second launchd job, `com.pulsecbd.recorder`, records the weekday peaks even when the server isn't running (files tagged `rec`).

- It runs at 06:55 and 16:25 for 130 minutes each, so it covers each peak with five minutes either side.
- When the server is already polling, the recorder sees the server's "heartbeat" file (`data/recordings/.server_polling`) and stands aside, so the same peak isn't paid for twice. Its log then says "0 feed payloads received", which is normal.
- It takes over if the server stops.

```
python scripts/run_record.py --minutes 130          # one-off recording
python scripts/install_recording_schedule.py        # weekday 06:55 and 16:25 (installed)
python scripts/install_recording_schedule.py --uninstall
```

Where files go. Recordings are stored as `data/recordings/{mode}/{feed type}/{hour}.{serve|rec}.ndjson.gz`. Each line is one download, with its fetch time.

- Hours are in UTC, not Sydney time: `20261104T15` is 2 am on 5 November in Sydney.
- If both writers recorded the same hour, replay reads the larger file. Until 30 October it preferred the `rec` file, which hid most of the Thursday 29 October PM peak.

Sleep. The wake schedule (`scripts/wake_schedule.sh`) was never needed, because sleep is disabled.

12.3 Staying within the data quota (`engine/sources/budget.py`)

The server and the recorder each add every request they make to one shared count for the day (`data/api_usage.json`, locked so the two processes can't both write at once). The budget is set to 57,000, about 5% under the measured quota. How fast the server polls then depends on the time of day:

- **During peaks** (weekdays 06:55–09:05 and 16:25–18:35), it polls every 15 seconds: 15 requests per cycle, one per feed.
- **Between peaks**, it holds back the requests the rest of today's peaks will need, and spreads what's left evenly until midnight. The interval stretches from 15 seconds up to at most 600 seconds.
- **When the day's budget is spent**, it polls once every 600 seconds: a few hundred requests a day. The live view slows down instead of going dark.

Current usage and pace are under Team → System health on the tablet, and at `/health`.

12.4 Calibration: lining the image up with the model

Why it's needed. Without calibration, the projector page simply fits the footprint rectangle into the screen. On the real model, the projector's position and angle mean that rectangle won't land exactly on the model's streets.

First-time calibration:

1. Press **C** on the laptop, or **Start** under Team → Alignment on the tablet. Four or more draggable crosshairs appear on the projection.
2. Drag each crosshair onto a physical mark on the model whose latitude and longitude are known (a corner, a distinctive intersection).
3. Type in that mark's latitude and longitude.
4. Save. The server computes a homography (§3.3) and saves it to `data/calibration.json`. The projection switches to it immediately, and it survives restarts.
5. Press **V** for the verification overlay. Route lines are drawn bright; check they sit on the model's streets and rail lines before trusting the projection.

Small drift between weekly sessions is fixed with the **nudge** on the team panel: shift by 2 px or 20 px, and scale.

Corner fit (teammate's, merged 5 November) is the quick way to line the map up when the projector has moved or isn't square to the table: Team → Alignment → Corner fit, or **F**.

- Four handles appear on the footprint's corners (north-west, north-east, south-east, south-west).

- Drag each onto the matching corner of the model, and the whole map follows. It is a homography through those four points, so it also removes keystone (the trapezoid shape from a projector that isn't straight above).
- For fine moves, set "Arrows move" to a corner and use the arrow pad.
- **Reset corners** puts them back.

The nudge and the corner offsets are both saved in `data/projection_nudge.json` and survive restarts. Saving a new crosshair calibration resets both.

According to the teammate's README update of 5 November, the projection is now aligned on the physical model. The crosshair calibration itself is recorded as tested only with synthetic points (§15).

12.5 Weekly model session

1. Check the server is running; open both pages.
2. Team → verification overlay on. Do the corridors sit on the model's streets?
3. Small drift: nudge. The projector moved or isn't square: corner fit. Neither enough: recalibrate with at least four marks.
4. Try every layer on the model. Note:
    - whether colours read on the white model;
    - whether line widths are visible (the glow that hid small drift is gone);
    - zones versus grid;
    - whether Full map labels are readable.

12.6 Showcase-day checklist

1. **Laptop:** on **power**, hotspot on, tablet joined to it, `/control` open and **reloaded**.
2. **Projector page:** full-screen and reloaded. Status line off, or on to label replays.
3. **Alignment:** verification overlay check, nudge if needed, then turn the overlay off.
4. **Defaults to confirm:** background map Plain, Early buses on, no area layer on.
5. **Replay for the presentation:** Replay mode → day → AM or PM peak, at 25×. Good candidates are any weekday peak at 100%, such as Wednesday 4 November AM, Thursday 5 November AM, or Monday 26 or Wednesday 28 October AM or PM.
6. **Live:** use it only if Team → System health shows fresh feeds (a few seconds old) and the API budget isn't spent.
7. **After each demonstration, press Live now.** Polling and recording are paused during replays.

12.7 Troubleshooting

Symptom	Likely cause	What to do
Tablet header says "connecting…"; projector frozen	Server not running or restarting	`lsof -nP -iTCP:8765 -sTCP:LISTEN`; if nothing, `launchctl kickstart -k gui/$(id -u)/com.pulsecbd.server`; read the end of `data/recordings/server.log`
Tablet can't open the page at all	Not on the same network	Join the laptop's hotspot; use the address printed in the server log
A page looks like an older version	It has old code cached	Reload the page
Area buttons greyed out	Layers still building after startup, or data files missing	Wait a few seconds after a restart; check the log for "areas" errors

Symptom	Likely cause	What to do
Background map buttons greyed out	`data/basemap/basemap.json` missing	`python scripts/fetch_basemap.py` (or `--cached`), then restart
Vehicles stop moving while live	Wi-Fi dropped, or polling has slowed to save the quota	Team → System health: feed ages of minutes mean no data is arriving; a long polling interval means the server is pacing itself (outside peaks) or the day's budget is spent. Switch to a replay
A replay looks sparse	It is a partly recorded session	Check the coverage note under Recorded peak; choose a peak at 98–100%
The model shows a replay after the demo	Replays don't end by themselves; they loop	Press Live now
Colours are offset from the streets	Calibration drift	Nudge (Team → Alignment), or recalibrate
Server keeps restarting	It crashes at startup	Read `server.log`; run `python scripts/run_publish.py` in a terminal after uninstalling the job, to see the error

12.8 Maintenance commands

```
# refresh the timetable (then restart the server)
python scripts/run_phase1.py --force-refresh
# only if zones or postcodes change
python scripts/prepare_demand_data.py --poa-dir <ABS shapefile dir>
# background map from OpenStreetMap (--cached: rebuild from the saved download), then restart
python scripts/fetch_basemap.py [--cached]
# one recorded peak as three CSV files in data/exports/<peak>/ (about 1.5 minutes)
python scripts/export_peak.py 2026-11-04-am
```

12.9 Exporting a peak to CSV

For spreadsheets and report figures, any recorded peak can be exported as three CSV files: from the tablet (Display → Time → *Download a recorded peak*), or on the laptop with `python scripts/export_peak.py <peak>` (for example `2026-11-04-am`; add `--out <folder>` to choose where).

How it works. The peak is replayed through the same engine the projection uses, as fast as it will go, starting 10 minutes early so trips and stop visits already under way are tracked. In that one pass `engine/analysis/export.py` writes:

- **every vehicle update**;
- **every stop visit**;
- **each trip once at TfNSW's measuring point**, with its on-time state (what the Stats tab counts, §7.2).

Behind the tablet button. The tablet calls `/export`, which runs the script in a separate process, so the live display isn't slowed. It keeps the files in `data/exports/<peak>/`, so later downloads of the same peak are instant. It refuses a peak that is still being recorded.

Checked on the Wednesday 4 November AM peak:

File	Rows	Size
Vehicles	222,002	41 MB
Stop visits	24,083	4 MB
Trips on time	1,272	under 1 MB

- The export took 98 seconds.
- Every file is within Excel's row limit of about 1 million.
- The replay found 24,083 stop visits, against 24,151 in the live history recorded at the time: a 99.7% match.
- Because stop visits come from the replay, they exist for every recorded peak, including those before the live history began on 30 October.

Whole days aren't offered: a day of vehicle updates would be around 2 million rows, more than Excel can open. Every column is defined in Appendix D.

---

13. The code repository

The code is in a private GitHub repository shared by the team, `0rangeGoose/Civil3704-group-6`, known locally as the remote `group-repo`.

Branch	Where	What it holds
`week13`	local + GitHub	**Current, and the build to run (since 5 Nov evening).** Everything on `w12merge` up to `5676e3b`, plus the 5 Nov evening commit(s) below. Pushed as a separate branch at the owner's request; nothing else merged in
`w12merge`	local + GitHub	The owner's work merged with the teammate's (merge `3ab2a8a`, 30 Oct), plus everything up to `5676e3b`. On GitHub it also has the teammate's `3b32562` "gradient" (9:22 pm, 5 Nov), which `week13` doesn't include. The local copy has the 5 Nov evening commit but was never pushed
`yosephW12`	local + GitHub	The owner's work through 30 October, before the merge (`3bfd432`)
`integration`	GitHub, `67d4d9c`	The teammate's 11 commits of 29–30 October on top of the owner's work to 26 October. Fully contained in `w12merge`
`test`	local + GitHub	The owner's prototype as of 28 August
`main`, `mid-project-demo`, `test-pipline`, `Mid-project-demo-A-edits`	GitHub	The teammates' template and dashboard work

Commits since the 30 Oct merge (on both branches up to `5676e3b`; the last row is on `week13` only):

Commit	What it did
`6ee94d2`	Stop visits recorded only while live
`17ad8fc`	The server's launchd job
`6fcca03`	Background map (Plain, Detailed, Full)
`14b7b9c`	Street pooling, faint streets, direction modes
`95c722a`	Replay day picker and colour definitions
`b456295`, `2494941`, `e6d99d9`	Teammate (4–5 Nov): corner fit, merged into `w12merge`, and a README rewrite; also adds `pulse-cbd-context.md`, their handoff notes
`016fdcf`, `b3249e9`	Merge of the teammate's work with the rewritten report and the owner's README update (the teammate's README kept)
`74d6b12`	Interchange rings removed; report walk-through extended

Commit	What it did
`5676e3b`	Renamed to Inner Sydney Transit Projection Model; tablet fits phones. **Last commit with the street colours, BOAM and step 6 code**, for anyone who needs them back
5 Nov evening (latest)	CSV export; PIN, Opal 2016, pressure and street colours (with BOAM and step 6 code) removed; punctuality from published standards with per-trip Stats and the Early buses switch; hover highlights routes; this report

Which branch is the showcase build still has to be agreed with the teammate (§18). The laptop runs `week13`. The teammate's newest work, the gradient, is only on `w12merge`.

Clean-up before submission. The teammate committed:

- personal editor settings (`.claude/settings.json`, `.vscode/settings.json`);
- about 55 MB of PDFs and a spreadsheet in `template-repository-main/New folder/`, not stored with Git LFS;
- their individual report, at the repository root.

The data (not in Git). `data/` is ignored by Git: it's 28 GB and several files are over GitHub's 100 MB limit. On 5 November all of it was uploaded to a private **GitHub release**, `data-2026-11-05`, on the same repository: 32 files, 27.7 GB, everything up to about 6:20 pm that day.

- `core-data.tar.gz`**:** timetable databases, jobs and zones, Opal, BOAM, background map, analysis outputs, and a safe copy of the stop history.
- **Recordings:** one `.tar` per Sydney half-day, `recordings-<date>-am-half` and `-pm-half`.
- **Not uploaded:** `.env` (the API key) and the logs. The files were checked for the key before upload.
- **Unpacking:** run `tar -xzf core-data.tar.gz` and `tar -xf recordings-….tar` from the project folder; the release notes give the steps.
- **Bundling:** the upload script bundled and uploaded one file at a time and paused in peak windows. It was a scratch script and isn't in the repository. Uploading again means re-creating it, or using `gh release upload`.
- **Not in the release:** Thursday 5 November's PM peak and anything recorded since.
- **Licence:** check TfNSW's terms before sharing the data outside the team.

Security. A teammate's TfNSW API key is committed in `template-repository-main/src/file.env` (commit `d85b5e1`). The repository is private, but the key should still be regenerated. The owner's key is only in `.env`, which Git ignores; it appears in no commit, and this is checked before every push.

---

14. State of the laptop (5 November, 11 pm)

Item	State
Server	launchd job, live, running the `week13` code (last restarted 10:45 pm on 5 Nov). 149 vehicles at 10:53 pm. Open pages need a reload to pick up the code
Recorder	Installed; its runs stand aside while the server is polling (its log then says "0 feed payloads received", which is normal)
Sleep	Disabled (`sudo pmset -a disablesleep 1`). Undo after 6 November with `sudo pmset -a disablesleep 0`. Keep the Mac **on power**: it was found on battery on 3 November
API usage	About 57,000 on every full day from 29 October to 2 November, and on 4 November. 3 November: 39,663 (server down 11:30–18:33). 5 November: 53,700 by 10:53 pm

Item	State
Disk	65 GB free. Recordings 28 GB; stop visits 128 MB (919,987 visits); analysis 58 MB and BOAM 42 MB (no longer read); CSV exports 45 MB; background map 16 MB
Server log	`data/recordings/server.log`: 129 MB, never trimmed
Data release	GitHub release `data-2026-11-05` (private): 32 files, 27.7 GB, everything in `data/` up to ~6:20 pm on 5 Nov (§13)
Node.js	Installed via Homebrew (only for syntax checks)

Replayable sessions (share of the expected bus updates actually recorded):

Day	AM	PM
Thu 5 Nov	100%	100%
Wed 4 Nov	100%	99%
Tue 3 Nov	100%	lost (Mac shut down)
Mon 2 Nov	100%	98%
Sun 1 Nov	66%	66%
Sat 31 Oct	66%	66%
Fri 30 Oct	99%	100%
Thu 29 Oct	100%	99%
Wed 28 Oct	100%	100%
Tue 27 Oct	100%	86% (Wi-Fi drop ~5 pm)
Mon 26 Oct	100%	100%
Sun 25 Oct	84%	96%
Sat 24 Oct	100%	—
Sun 6 Sep	54%	99%
Sat 22 Aug	—	44%

Weekends read 66% because polling slows outside weekday peaks to save the quota. "Latest 2 hours" is also always available.

Known outages:

- Wi-Fi drops: 23 October from about 4:31 pm (most of that PM peak lost), the afternoon of 25 October, and 27 October from 5:00 to 5:20 pm and from 7:40 to 10:42 pm.
- TfNSW throttled the key from 2 to 4 pm on 28 October (not our quota).
- The Mac shut down after 11:30 am on 3 November, and the server stayed down until 6:33 pm.

Part V — Assessment

15. Limitations and caveats

These should be stated plainly in the final report.

Data

- **TfNSW's fields are undocumented categories.** Neither the traffic nor the crowding field has published numbers behind it. Crowding is almost always "seats available" off-peak. Light rail and ferries report neither field. Sydney Trains' traffic is always unknown, and about 70% of Sydney Trains have no usable delay.
- **Recordings have gaps** wherever Wi-Fi dropped (§14).

Method

- **Relative layers.** The service gap and the jobs fifths are relative to the footprint, not measured against planning standards. Jobs fifths are affected by zone size: a large zone can rank high just for being large. The grid avoids this.
- **Departures measure reachable service.** A trip counts once per stop within reach, not once in total.
- **Punctuality limits are partly borrowed.** Trains and buses use published TfNSW standards, but Metro, light rail and ferries have no published definition, so they borrow the Sydney Trains 5-minute standard. "Missed its slot" is the project's own category, and can't apply to Sydney Trains, Metro or CBD light rail (no timetable match). The live colour is a prediction at the next stop; official-style figures (Stats tab) use predicted arrivals at final stops for trips ending outside the model (§7.2).
- **Our congestion measure (no longer in the code) was limited.** It uses a night baseline (fewer passengers, so every peak street reads somewhat slower). 38% of peak readings have no baseline. Its agreement with TfNSW is weak (§10).

Setup

- **Limited testing on the physical model.** The teammate reports the projection aligned on the model with corner fit; checks of every layer there aren't recorded. Crosshair calibration is recorded as tested only with synthetic points. The glow that used to hide small drift is gone.
- **Replays aren't labelled.** The projection doesn't say it is showing a replay unless the status line is on.
- **No access control.** Since the team PIN was removed (5 November), anyone at the tablet can change alignment and calibration, and the server accepts connections from any device on the network (use the hotspot).
- **One laptop.** Replays depend on one laptop and one disk, and the server log is never trimmed.
- **Untested page code.** The projector and tablet pages have no automated tests.

---

16. Remaining work, in priority order

Before the showcase (Friday 6 November)

1. **Reload the projector and tablet pages,** if any were open before 10:45 pm on 5 Nov, so they run the latest code.
2. **Decide about the mouse pointer on the projection.** The projector page is meant to hide it, but the teammate turned that off on 29 Oct for testing (`renderer/index.html`: the commented-out `body { cursor: none; }`, marked "TEMPORARILY disabled"). The pointer is projected onto the model, but hover features need a pointer to aim with.
3. **Finish checking on the physical model** (alignment is reported done by the teammate):
    - vehicle colours, including blue Early buses;
    - stop markers and stop history;
    - jobs and service gap on zones versus grid;
    - the hover highlight;
    - Full map label size.

4. **Test the networking.** A Mac can't run a hotspot and join Wi-Fi at the same time, so live mode needs one of:

    - Ethernet;
    - USB tethering to a phone;
    - a phone hotspot that both the laptop and the tablet join.

    If none works, run replays only; they need no internet.

5. **Agree with the teammate** that `week13` is the showcase build (and whether to bring in their gradient from `w12merge`), and that about 60–70% of Sydney Trains show grey (no live prediction).

6. **Pick the showcase replay** (§12.6), and rehearse the tablet flow: Replay mode → day → peak → **Live now** afterwards.
7. **Make it clear when a replay is showing:** status line on, or say so out loud.
8. **Make the report figures:**
    - the per-trip on-time figures (Trips CSV);
    - jobs and the service gap;
    - the step 6 tables from §10;
    - the colour table in §8.
9. Ask the teammate to **regenerate the API key they committed.**

Done on 5 November: the PM peak was recorded (100%), everything in `data/` was backed up to the GitHub data release, and the server was restarted onto the latest code.

Optional or undecided

- Add an alert on the Mac when Wi-Fi stalls (offered, not built).
- Find official on-time definitions for Metro, light rail and ferries (operator contracts, or ask TfNSW) to replace the borrowed 5-minute standard.
- Trim or rotate `server.log`.

After the showcase: `sudo pmset -a disablesleep 0`; `python scripts/install_server_service.py --uninstall`; `python scripts/install_recording_schedule.py --uninstall`.

---

17. Suggestions for improvement

1. **Guided demo mode:** one button plays a scripted tour (AM peak at 25× → vehicle traffic → crowding → service gap), with captions on the tablet.
2. **A simpler visitor tab:** move the grid switch, the Early buses switch and the background map behind a "More" button. The legend gets long when several layers are on.
3. **A replay label on the projection itself:** the date and time in a corner, independent of the status line.
4. **Check the service gap against an accessibility measure,** such as jobs reachable within 30 minutes, for sample zones. Also test how sensitive it is to the walking distance (400 m instead of 200 m).
5. **Faster startup,** by caching route corridors like the area layers.
6. **Browser tests** for the tablet and projector pages, now that the legend, replay picker and hover highlight hold real logic.
7. **Touch markers on the model** at stations (from the original brief; out of scope this term).

(Suggestions about the street layers, BOAM and re-running the step 6 analysis were dropped when those were removed on 5 November.)

---

18. Open decisions waiting on the owner

1. Confirm `week13` as the showcase build with the teammate, and decide about their gradient commit on `w12merge`.
2. Which peak is the showcase's default replay?
3. Build the Wi-Fi stall alert?
4. Show the mouse pointer on the projection (needed for hover), or hide it?

---

19. Timeline

Date	What happened
August	Prototype built in six phases on `test`: data engine, projector page, calibration, statistics
22–23 Aug	Proposal and timeline documents
25 Sep	Design session; plan, decisions and build order agreed (§2.3)
23 Oct	Steps 1–3: footprint, tablet controller, recording, street traffic and crowding, replay. The rate-limiting problem and slow map matching fixed. The first scheduled PM peak lost to Wi-Fi
23 Oct	Steps 4–5: area layers and pressure. BOAM imported (5 days) and used three ways. Branch `integration` pushed
24–25 Oct	Server run continuously. Quota exhausted at 7 pm on Sunday, which is how the quota was measured
26 Oct	Quota-aware polling and 30-second network retries. The first complete weekday peaks
29–30 Oct	The teammate pushes 11 commits to `integration`
30 Oct	Replay pause and seek. Timetable refreshed. Step 6: new method, experiment, final analysis. 12 more BOAM days (school term). Work pushed to `yosephW12`. Replay file-choice fix (Thursday 29 October PM from 50% to 99%). Merged with `integration` as `w12merge`. Stop visits made live-only
3 Nov	Mac shut down after 11:30. Server down until 18:33 and the PM peak lost. Server moved to a self-restarting launchd job
4 Nov	Optional background map (Plain, Detailed, Full). Street glow removed. Streets pooled, faint when thin; long BOAM sections left out of time lost. Street direction modes, with one line per stretch
4–5 Nov	Teammate: corner fit for aligning the projection on the model (F), merged into `w12merge`; README rewrite
5 Nov	Renamed from Pulse CBD to the Inner Sydney Transit Projection Model (pages, tablet, README, report). Tablet: Replay mode with a day dropdown and AM/PM buttons; every colour defined with its cut-off, read from the server; stop-history legend. This report rewritten for readers without prior context, and the teammate's work merged in. Interchange pulses removed from the projection. Team tab PIN removed. Opal 2016 removed (area layer and pressure input), then the pressure layer itself. Street colours removed, with their engine code, BOAM and the step 6 analysis code (results kept in §10). CSV export of any recorded peak (tablet Display → Time, `scripts/export_peak.py`)

---

20. Key files

Area	Files
Configuration	`config/config.yaml` (Appendix C)
Fetching and recording	`engine/sources/live.py` (polling), `budget.py` (quota), `recorder.py`, `footprint.py` (trimming), `replay.py` (sessions and playback)
Processing	`engine/normalise/build.py`, `enrich.py` (joining to the timetable), `map_match.py`, `adherence.py`, `trip_updates.py`, `levels.py`, `headways.py` (first stops and scheduled gaps), `aggregate.py` (statistics, per-trip punctuality, interchanges); `engine/analysis/export.py` (the CSV export)
Static layers	`engine/demand/areas.py` (jobs, departures, service gap)
Server	`engine/publish/server.py` (addresses), `feed.py` (live state), `control.py` (display state), `calibration.py`, `homography.py`, `route_focus.py`, `shapes_export.py`
Stop history (teammate)	`engine/history/stop_tracker.py`, `store.py`
Pages	`renderer/index.html` (projector), `renderer/controller.html` (tablet)
Scripts	`run_publish.py` (server), `run_record.py`, `install_server_service.py`, `install_recording_schedule.py`, `fetch_basemap.py`, `run_phase1.py` (timetable), `prepare_demand_data.py`, `export_peak.py` (CSV export). The BOAM and step 6 analysis scripts were removed on 5 Nov
Tests	`tests/` (173 tests; e.g. `test_adherence.py`, `test_headways.py`, `test_areas.py`, `test_control.py`, `test_replay.py`, `test_basemap.py`)
Data (not in Git)	`data/static_cache/` (timetable), `data/recordings/`, `data/demand/`, `data/opal/` and `data/boam/` (no longer read), `data/analysis/`, `data/history/` (stop visits), `data/basemap/`, `data/api_usage.json`, `data/projection_nudge.json`, `data/calibration.json`
Documents	`README.md`, `docs/DECISIONS.md`, `docs/PROJECT_REPORT.md`

Appendix A. Display settings (`engine/publish/control.py`)

Every setting returns to its default when the server restarts, except the nudge and the corner-fit offsets, which are saved to `data/projection_nudge.json`.

Setting	What it controls	Allowed values	Default
`theme`	Projection colours	dark, light	dark
`basemap`	Background map	plain, detailed, full	plain
`verify`	Verification overlay	on / off	off
`hud`	Status line on the projection	on / off	off
`calibrate`	Calibration crosshairs	on / off	off
`early_buses`	Buses running early drawn blue (on) or as on time (off)	on / off	on
`stops`, `stop_history`	Stop markers; stop history colours	on / off	off
`stop_metric`	Stop history delay statistic	median, mean	median
`routes`	Routes singled out by the filter	up to 8 route names	none
`route_slots`	Colour assigned to each filtered route	0–7, set by the server	—
`lens`	What vehicle colour shows	adherence, congestion, crowding	adherence
`area_layer`	Area layer	none, jobs, service_gap	none
`area_unit`	Area shape	zones, grid	zones

Setting	What it controls	Allowed values	Default
`playback`	Live or a session (e.g. `2026-11-04-am`), speed, paused	live or a recorded session; 1–100×; yes/no	live, 25×, not paused
`modes`	Vehicle groups: trains, metro, lightrail, buses, ferries	on / off each	all on
`nudge`	Alignment correction	dx and dy ±2000 px; scale 0.5–2	0, 0, 1
`corners`	Corner-fit handles shown	on / off	off
`warp`	Corner-fit offset of each footprint corner (tl, tr, br, bl)	x and y ±2000 px each	all 0

Appendix B. Server addresses (`engine/publish/server.py`, port 8765)

Address	What it returns
`/`	The projector page
`/control`	The tablet page (`/stats` redirects to its Stats tab)
`/ws`	The WebSocket: vehicle positions about twice a second and the display state; accepts setting changes
`/config`	Footprint, polling timing, mode groups, level labels, punctuality and TTI thresholds, whether a background map exists
`/health`	Vehicles per mode, feed ages, API use today, polling interval, playback
`/snapshot`	Every current vehicle
`/shapes`	Route lines for drawing
`/areas`	Zones and grid squares with values and colour bins (`?meta=1`: availability, build details, per-fifth ranges)
`/stops`, `/stop_history`	Stop markers, each with the routes serving it (built once, ~2.5 s); per-stop delay and crowding summaries
`/routes`, `/route`	The route list for the filter; one route's line and stops
`/recordings`	Replayable sessions with coverage
`/basemap`	The offline map layers and labels
`/aggregate`	The Stats tab's figures
`/calibration`	Read the calibration; send at least 4 points to save a new one
`/export`	One recorded peak as CSV: `?session=2026-11-04-am&kind=vehicles` (or `stop_visits`, `trips`). Makes it the first time, about 2 minutes (§12.9)

Appendix C. Configuration (`config/config.yaml`)

Section	What it sets
`bounding_box`	The footprint corners (§2.2)
`calibration_path`	Where the calibration is saved
`static_gtfs`	Timetable download address and cache location
`realtime_gtfs`	The 15 live feeds polled (7 vehicle-position, 7 trip-update, 1 combined alerts), the polling interval, and `stale_after_seconds` (90 s: how long without an update before a vehicle is treated as gone)
`punctuality`	Per-mode on-time limits, late and (buses) early, each with its source (§7.2)

Section	What it sets
`interchanges`, `interchange_radius_m`	The nine interchanges and the 300 m radius counted for the Stats tab (§7.3)
`server`	Listens on every network interface, port 8765
`api_budget`	Daily limit 57,000, the peak windows, the 600 s slowest interval, file locations (§12.3)

Appendix D. CSV export columns (`engine/analysis/export.py`)

All times are Sydney time, written as ISO dates with the time-zone offset (e.g. `2026-11-04T07:00:08+11:00`). Blank means no value.

vehicles.csv: one row per vehicle per feed update (about every 15 seconds), including updates where the vehicle hasn't moved.

Column	Meaning
`time`	When the update was downloaded (the engine's clock)
`vehicle`	Mode and TfNSW's vehicle ID, e.g. `buses:43264_…`
`mode`, `route`, `headsign`, `trip_id`	Feed mode, route name (e.g. 333, T1), destination shown, timetable trip
`lat`, `lon`	Position snapped to the route line (§7.1), or the raw GPS if it couldn't be placed on one
`bearing`	Direction of travel in degrees from north, as reported. Sydney Trains send none, so theirs comes from the direction of the route line where the trip is matched to it; blank otherwise (10% of train updates on 4 Nov AM)
`delay_s`	TfNSW's predicted delay at the next stop, in seconds; positive is late (§7.2)
`on_time_state`	`early`, `on_time`, `late`, `very_late` (missed its slot), `cancelled` or `unknown` (§7.2)
`tfnsw_traffic`	TfNSW's `congestion_level` for the vehicle (§7.4); buses only in practice
`tfnsw_crowding`	TfNSW's `occupancy_status` (§7.4); the middle carriage for trains
`position_reported_at`	The vehicle's own timestamp for that position

stop_visits.csv: one row each time a vehicle passed a stop (§7.13), worked out from the replay.

Column	Meaning
`time`	When the replay saw the vehicle had passed the stop
`stop_id`, `stop_name`, `stop_lat`, `stop_lon`	The stop, from the timetable
`route`, `mode`, `vehicle`	Which vehicle passed
`delay_s`	Its delay as it left the stop, in seconds; blank where TfNSW gave no real prediction
`tfnsw_crowding`	Its `occupancy_status` as it left

trips.csv: one row per trip, measured once at TfNSW's measuring point (§7.2); what the Stats tab counts.

Column	Meaning
`time`	When the reading was taken (the last one wins)
`trip`	Mode and TfNSW trip ID
`mode`, `route`	Feed mode and route name

Column	Meaning
`measured_at`	`first stop` (buses, as they leave it) or `final stop (predicted)` (every other mode)
`stop_id`, `stop_name`	The stop measured at
`delay_s`	Delay there in seconds; positive is late
`state`	`early`, `on_time`, `late` or `cancelled` by the mode's on-time limits (no "missed its slot" here: TfNSW's figures are on time or not)